

MacPorts ports tree development notes

l2dy

Contents

1. Beginner's Guide	4
1.1. Introduction to port phases	4
1.2. Portfile keywords and variables	4
1.3. Build and destroot directory structure	4
1.4. Dependency types	4
1.5. Fetching distfiles	4
1.6. Variants and subports	4
2. Conventions	5
2.1. Only bump revision if binary archive contents would change	5
2.2. Use PortGroups where possible	5
2.3. Use system or MacPorts-provided libraries where possible	5
2.4. GitHub distfiles	5
2.5. Use -append where possible	6
2.6. Disable undeclared dependencies in configure arguments	6
2.7. Use path:-style specifier for dependencies when alternatives exist	6
2.8. How to declare port conflicts properly	6
2.9. Obsoleting a port	6
2.10. Disable auto-updater	6
2.11. Case-sensitive file system support	7
3. Tricks	8
3.1. Portfiles may contain Tcl code	8
3.2. Copying extra files in post-destroot	8
3.3. Handling non-standard distfile name, suffix, directory structure and alternative compression method	8
3.4. Multi-line port notes	9
3.5. License and missing binary archives	9
3.6. Sharing a distfiles cache and mirror path	9
3.7. Dealing with stealth update of distfiles	9
3.8. Dummy master_sites URLs	10
3.9. Checkout git submodules	10
3.10. Patching source files	10
3.11. Patching configure.ac	11
3.12. Skipping phases and re-enabling them	11
3.13. Build without configure phase	12
3.14. Glob matching files	12
3.15. Default configuration files	12
3.16. Multiple distfiles	12
3.17. Conflicting variants with a default variant	13
3.18. Conditional evaluation based on OS version	14
3.19. Compile new C++ code on old OS versions	14
3.20. The legacysupport PortGroup	14
3.21. Declaring incompatibility with older macOS releases	14
3.22. The conflicts_build PortGroup	14

3.23. The select PortGroup	14
3.24. Additional CMake module path	15
3.25. Override CMAKE_BUILD_TYPE	15
4. Upstream build fixes	16
4.1. Prioritize LUAJIT_INCLUDE_DIR over default search path in CMake	16
4.2. Hard-coded paths in binary	16
4.3. Respect DESTDIR in Makefile correctly	16
4.4. Fix redefinition of library identifiers	17
4.5. Add missing #include	18
4.6. Remove -Werror from build flags	18
4.7. Sequential build with -j 1	19
4.8. Preserve specific empty directories in destroot	19
4.9. Fix file permissions in extracted tarballs	19
5. Tips	20
5.1. Livecheck and Repology	20
5.2. Force building from source	20
5.3. Verify patches quickly	20
5.4. Create a patch file from git commands	21
5.5. Install a subport without re-indexing	21
5.6. Switch between conflicting dependencies	21
5.7. Identify library dependencies	21

1. Beginner's Guide

This chapter describes what is important to know before writing a Portfile for ports tree inclusion. It complements the Portfile development section of the MacPorts Guide[°], so you should read the official guide too.

1.1. Introduction to port phases

Please read the port phases section of Portfile reference in the MacPorts Guide[°].

1.2. Portfile keywords and variables

Please read the relevant sections in the MacPorts Guide[°] *as needed*.

For example, if you read the Portfile reference before using the `version` keyword, you'll find out that it's recommended to strip any `v` prefix in the port version.

1.3. Build and destroot directory structure

You can `destroot` a stub port to get an idea of the directory structure:

```
sudo port -v destroot fennel_select
find "$(port work fennel_select)"
```

And you will find out why some ports would set `--mandir` in `configure.args` like so:

```
configure.args      --mandir=${prefix}/share/man
```

The `destroot` directory structure matches the file hierarchy for `${prefix}`, which is explained in detail in the MacPorts Guide[°].

You can also `patch` a real port to understand how its `${worksrcpath}` is prepared:

```
sudo port -v patch git
```

After you are done with the experiments, you can clean up the work directories with either `port reclaim` or `port clean <port>`.

1.4. Dependency types

Please read the dependencies section of Portfile reference in the MacPorts Guide[°].

1.5. Fetching distfiles

See Section 2.4 and Section 3.3. You should also refer to the fetch phase keywords section in the MacPorts Guide[°] for more information.

1.6. Variants and subports

Variants and subports are relatively advanced features. You should read the MacPorts Guide[°] and some well-written ports in the ports tree to understand how to implement them.

2. Conventions

This chapter describes some of the best practices in the MacPorts ports tree. It complements the MacPorts Guide[°].

2.1. Only bump revision if binary archive contents would change

You should increase the revision if and only if it would result in a change for users who already have the ports with the same version installed. This is also known as a revision bump, or revbump for short.

Some examples where a port's revision should usually be increased:

1. Change of files installed by a port. This includes incompatible ABI changes in one of its dependencies (commonly seen when a library port gets updated) that demands a rebuild, new patches or destroot changes.
2. Adding a variant name to `default_variants`, and removing existing variants.

More examples are given in the MacPorts Guide[°]. A notable exception to the aforementioned examples is fixing a build failure, because no users can have the broken port version installed.

If the version or epoch has been increased, you should reset the revision to 0.

2.2. Use PortGroups where possible

PortGroups simplify your Portfiles, and help enforce conventions that matches best practices in MacPorts. You should review the current list of PortGroups in `_resources/port1.0/group` before implementing a new port.

Some PortGroups are documented in the MacPorts Guide[°],

There are also helper CLIs like `go2port` and `cargo2port`, which can be installed with MacPorts.

2.3. Use system or MacPorts-provided libraries where possible

Always prefer using other MacPorts ports as library dependencies instead of building bundled third-party libraries in the port's build process.

Many Linux distros also enforce this policy. If you are interested, you can read the embedded code copies[°] section from the Debian Policy Manual, and <https://wiki.debian.org/Packaging/EmbeddedCopies>[°] for some trade-offs.

2.4. GitHub distfiles

In the `github 1.0` PortGroup, `github.tarball_from tarball` is the default but has been deprecated. It's recommended to add the following after `github.setup`:

```
github.tarball_from archive
```

If upstream uploads complete source tarballs to GitHub releases, those are preferred to avoid stealth distfile changes in git-generated tarballs:

```
github.setup      translate translate 2.4.0
github.tarball_from releases
distname          translate-toolkit-${version}
```

`distname` is needed if the released filenames contain tag prefixes like `v` or don't match the GitHub project name.

2.5. Use `-append` where possible

Using `-append` enables composition of multiple sets of values through the Portfile, PortGroups and MacPorts system's defaults.

For example, using `depends_build-append` instead of `depends_build` avoids overriding the `cmake` dependency declared in the `cmake 1.1` PortGroup.

2.6. Disable undeclared dependencies in configure arguments

If an optional dependency not declared in the Portfile is installed on a user's system, a source build may result in a opportunistically dependency relationship with the library port.

This is undesirable because uninstalling the library port would break the locally built port, therefore you should either explicitly disable the feature via `configure.args --disable-...` or include the dependency as `depends_lib`. See commit 2df55b8^o for an example.

2.7. Use `path: -style` specifier for dependencies when alternatives exist

For example, both `openssl` and `libressl` provides the `libssl` library, and they conflict with each other. To ensure `libressl` can be used as an alternative, you need to declare dependency on OpenSSL as follows:

```
depends_lib-append path:lib/libssl.dylib:openssl
```

2.8. How to declare port conflicts properly

For each new port installed, MacPorts only checks its `conflicts` field against active ports. Therefore, when two ports conflict with each other, you must define `conflicts` on both sides.

2.9. Obsoleting a port

When obsoleting a port, always use the `obsolete 1.0` PortGroup and add a comment before `replaced_by` dated a year after the commit is made:

```
PortGroup          obsolete 1.0

# port to be removed by 2024-05
replaced_by        xxx
```

You should keep the port name and version intact and bump the revision by 1. Remove all other code in the Portfile and the `files/` directory if it exists.

2.10. Disable auto-updater

Upstream's auto-update mechanism often conflicts with MacPorts' requirements to track the port's version and file list. Therefore, it's a common practice to modify upstream code to disable its auto-update function. For example:

```
reinplace "s|\"disable_updater\": false|\"disable_updater\": true|"  
${worksrcpath}/lib/googlecloudsdk/core/config.json
```

2.11. Case-sensitive file system support

The official MacPorts Buildbot builders use case-sensitive file systems, so watch the Buildbot results in case something only goes wrong on case-sensitive file systems.

3. Tricks

This chapter describes common patterns for handling tricky ports.

3.1. Portfiles may contain Tcl code

For example, You may define variables with Tcl's `set` command and use them later:

```
version            0.3.8
set branch         [join [lrange [split ${version} .] 0 1] .]
master_sites       https://www.example.com/download/Source/${branch}/
```

or use `foreach` to implement a loop:

```
foreach bin [glob -tails -directory ${destroot}${libexecdir}/bin pub dart?*] {
    xinstall -m 0755 ${filepath}/shim.in ${destroot}${prefix}/bin/${bin}
    reinplace "s|@@BIN@@|${libexecdir}/bin/${bin}|g" ${destroot}${prefix}/bin/
    ${bin}
}
```

where `shim.in` is a template file:

```
#!/usr/bin/env bash
exec "@@BIN@" "$@"
```

You could also use regular expressions for advanced string manipulation:

```
set distfolder      [regsub {[^\.]}\.([^\.]+)\.([^\.]*)p.*} ${version} {\1-
\2-\3}]
```

3.2. Copying extra files in `post-destroot`

You may want to install documentation alongside the programs built:

```
post-destroot {
    set docdir ${prefix}/share/doc/${name}
    xinstall -m 755 -d ${destroot}${docdir}
    xinstall -m 644 -W ${worksrcpath} LICENSE README.md ${destroot}${docdir}
}
```

This could be adapted to copy other files too, but check the existing directories in `destroot` first to determine if you still need the `xinstall -m 755 -d` command.

3.3. Handling non-standard distfile name, suffix, directory structure and alternative compression method

The default `extract.suffix` and `extract.cmd` expects a `.tar.gz` tarball. If the actual suffix is `.tgz`, you could override `extract.suffix`:

```
extract.suffix      .tgz
```

For alternative extract methods and their default suffixes, see the `use_*` keywords listed in the `extract` phase section of the MacPorts Guide^o.

If the distfiles name is not `${name}-${version}${extract.suffix}`, you can either override `distname` or `distfiles` itself. Consider the expected `worksrcdir` to determine which approach is cleaner because `worksrcdir` defaults to `${distname}`.

`worksrcdir` is the path to source directory relative to `workpath`. You can override it if the extracted source directory has a different name than the distfile, or if the project to be built is in a subdirectory of the extracted distfile:

```
worksrcdir      ${distname}/src
```

3.4. Multi-line port notes

Write multi-line port notes like so:

```
notes "  
...  
...  
"
```

In Tcl, `{ }` prevents variable, command and backslash substitution, so:

```
notes {  
    echo \n "${PATH}"  
}
```

would be printed verbatim:

```
$ port notes xxx  
---> xxx has the following notes:  
    echo \n "${PATH}"
```

Tips: `{ }` could also be used to simplify regular expressions, otherwise `\` has to be written as `\\` in `livecheck.regex` and `reinplace`.

3.5. License and missing binary archives

`license` determines if a port's binary archives are distributable on MacPorts' archive mirrors, so it's important to get it right. Always double check if it's correct when you copy another port's Portfile as a template.

When a port is considered as not distributable, it will not be uploaded to MacPorts' archive mirrors. You may run `port_binary_distributable.tcl` to get a port's distributable status and the reason.

If you believe that license conflicts with a dependency port is a false positive, you may set `license_noconflict` to manually override the port's distributable status.

3.6. Sharing a distfiles cache and mirror path

You can share distfiles across port to deduplicate these files on the distfiles mirror and your local cache by specifying a common `dist_subdir`. For example:

```
dist_subdir      ruby
```

3.7. Dealing with stealth update of distfiles

Stealth update of distfiles means that content of the same distfiles downloaded from it's source of truth (`master_sites`) have changed.

In this situation, files in MacPorts' distfiles mirror and local cache becomes inconsistent with a fresh download, and checksum mismatch with the Portfile is bound to happen either way.

To workaround this, you can use `dist_subdir` to specify a new directory for the mirror and local cache. For example:

```
# Stealth update, remove on the next version update
dist_subdir      ${name}/${version}_1
```

Alternatively, if the old distfiles have been saved by the MacPorts distfiles mirror, you could also ignore the stealth update completely:

```
# Ignore stealth update of 1.x.x
master_sites     macports_distfiles
```

3.8. Dummy `master_sites` URLs

If the download URL cannot end with distfile names, you can append `?dummy=` and specify which file to download in the middle of the `master_sites` URL. For example:

```
master_sites     https://code.monotone.ca/p/monotone/source/download/
                  ${commit}?dummy=
```

3.9. Checkout git submodules

If upstream doesn't provide a complete tarball and submodule checkout is required for the build, use:

```
fetch.type       git

post-fetch {
    system -W ${worksrcpath} "git submodule update --init --recursive"
}
```

The situation may improve if <https://trac.macports.org/ticket/50708>^{*} is implemented.

3.10. Patching source files

You may use `patchfiles` to apply unified diff patches or `reinplace` in `post-patch` phase for simpler regex replacements.

For `patchfiles`, It's common to name the files `patch-relative-path-to-file.diff` or `patch-xxx.diff` (replace `xxx` with summary or keywords of the patch.)

You could also combine the two approaches by adding `@@PREFIX@@` placeholders in patch files and put `${prefix}` into the patched files in `post-patch` phase with `reinplace`. For example:

```
patchfiles       patch-deps-tool-path.diff

post-patch {
    reinplace -W "${worksrcpath}" "s|@@PREFIX@@|${prefix}|g" \
        deps/build_deps.sh
}
```

along with the following patch file:

```

--- deps/build_deps.sh
+++ deps/build_deps.sh
@@ -63,9 +63,9 @@ SDKROOT = $SDKROOT
MACOSX_DEPLOYMENT_TARGET = $MACOSX_DEPLOYMENT_TARGET
CONFIGURATION = $CONFIGURATION

-sed = $GSED
-yacc = $YACC
-curl = $CURL
+sed = @@PREFIX@@/bin/gsed
+yacc = @@PREFIX@@/bin/yacc
+curl = @@PREFIX@@/bin/curl

top_srcdir = `pwd`
builddir = $DEPS_BUILD_DIR

```

Tips: be careful to restore the `@@PREFIX@@` parts if you are facing a patch conflict and use `diff` to generate a new patch. Otherwise you'll be hard-coding your local prefix into the patch file.

`reinplace` can also be used in the `post-destroot` phase to patch the build outputs directly, but it's generally not recommended unless patching the source files to implement the same effect would be too difficult to maintain.

3.11. Patching `configure.ac`

After patching `configure.ac` or `Makefile.am`, you need `autoreconf` to update the corresponding `configure` and `Makefile.in` files. To run it before `configure` phase, add the following in your Portfile:

```
use_autoreconf      yes
```

This is also needed if the `configure` script is missing, e.g. in a git checkout. Using proper source tarballs usually obviates this, but `-devel` ports may not have the chance to.

3.12. Skipping phases and re-enabling them

While not commonly seen, you can skip the `configure` and `build` phases and rely on `destroot` only:

```

use_configure      no

build {}

destroot {
    ...
}

```

If you still need the `build` phase in a subport or under an `if` condition, use:

```

build {
    command_exec build
}

```

It's also possible to have a stub port that doesn't produce any meaningful build output. The `stub` PortGroup implements this cleanly.

3.13. Build without configure phase

You should use the `makefile` PortGroup for a Makefile-only project.

In case you are curious how this was done before the PortGroup was implemented, here is a Portfile snippet that is sufficient for simple Makefile builds without `+universal` variant support:

```
use_configure      no

build.args-append  CC=${configure.cc} \
                   CFLAGS="${configure.cflags}" \
                   LDFLAGS="${configure.ldflags}"

destroot.args      prefix=${destroot}${prefix}
```

3.14. Glob matching files

`{*}[glob ...]` expands the files matched by `glob` (supports `?`, `*`, `[chars]` and `{a,b}` syntax) into multiple arguments, perfect for use in `xinstall`, `move` and `delete` commands:

```
xinstall -m 0755 \
  {*}[glob ${worksrcpath}/target/[cargo.rust_platform]/release/
  ${name}-{collapse-dtrace,collapse-ghcprof,collapse-guess,collapse-perf,collapse-
  recursive,collapse-sample,collapse-vsprof,collapse-vtune,diff-folded,flamegraph}]
  \
  ${destroot}${prefix}/bin/
```

3.15. Default configuration files

To avoid overriding users' custom configuration during port upgrades, it's common to rename the configuration files appending a suffix in `post-destroot`, and only copy the file to its expected destination if the file doesn't already exist:

```
post-destroot {
  move ${destroot}${prefix}/etc/xxx.conf ${destroot}${prefix}/etc/
  xxx.conf.sample
}

post-activate {
  if ![file exists ${prefix}/etc/xxx.conf] {
    copy ${prefix}/etc/xxx.conf.sample ${prefix}/etc/xxx.conf
  }
}
```

However, this also means that the user is responsible for merging new upstream defaults manually.

3.16. Multiple distfiles

When multiple distfiles are defined, each `checksums-append` block must list the distfile name before its checksums and size, and `master_sites` and `distfiles` should be tagged with `:<tag>` if there are multiple `master_sites` to download from. The same rules apply if any `patchfiles` need to be downloaded from `master_sites`.

Take `ashuffle` as an example:

```
github.setup      joshkunz ashuffle 3.13.3 v

github.tarball_from archive
master_sites      ${github.master_sites}:ashuffle
distfiles         ${distname}${extract.suffix}:ashuffle

checksums-append  \
                  ${distname}${extract.suffix} \
                  rmd160 3143e83c0fcbecf2c9cd9165cb67c2e5091649c4 \
                  sha256
e324409280bb07e5b15e250197c3c115cdccb5de801a8ded6bdfef0ea89cb006 \
                  size 85824

# BEGIN abseil (requires C++17 build)
set abseil_project abseil-cpp
set abseil_version 20211102.0

master_sites-append https://github.com/abseil/${abseil_project}/
archive/${abseil_version}:abseil
distfiles-append    ${abseil_project}-${abseil_version}${extract.suffix}:abseil
checksums-append    ${abseil_project}-${abseil_version}${extract.suffix} \
                  rmd160 bca4a16eaab1602cdc7ace8dd1ff82467b71b59e \
                  sha256
dcf71b9cba8dc0ca9940c4b316a0c796be8fab42b070bb6b7cab62b48f0e66c4 \
                  size 1884080

...
# END abseil
```

3.17. Conflicting variants with a default variant

If a recommended feature is provided by several conflicting variants, you could use `if` and `variant_isset` to conditionally set `default_variants`:

```
variant gdbm conflicts lmbd db4 description {Use GNU dbm database} {
    ...
}

variant lmbd conflicts gdbm db4 description {Use LMDB database} {
    ...
}

variant db4 conflicts gdbm lmbd description {Use Berkeley DB database} {
    ...
}

if {[variant_isset gdbm] && ![variant_isset lmbd] && ![variant_isset db4]} {
    default_variants-append +gdbm
}
```

This simplifies using other variants like `+db4`, and still allows the feature to be disabled explicitly with `-gdbm`.

3.18. Conditional evaluation based on OS version

To apply specific patches based on OS version, use:

```
if {{os.platform}} eq "darwin" && {{os.major}} >= 22} {  
    ...  
}
```

3.19. Compile new C++ code on old OS versions

`compiler.cxx_standard` is a MacPorts base feature that automatically introduces additional build dependencies on older OSes and sets `configure.cxx` accordingly:

```
compiler.cxx_standard 2017
```

3.20. The `legacysupport` PortGroup

The `libMacportsLegacySupport` library provides compatibility functions and declarations for older OS X releases. To check what features are supported, visit <https://github.com/macports/macports-legacy-support>^o.

The `legacysupport` PortGroup integrates it into configure flags automatically, so you only need to include the PortGroup:

```
PortGroup          legacysupport 1.1
```

3.21. Declaring incompatibility with older macOS releases

If a port is known to fail to build on older macOS releases, you should use `known_fail` to abort build attempts early. For example:

```
if {{os.platform}} eq "darwin" && {{os.major}} < 18} {  
    known_fail      yes  
    pre-fetch {  
        ui_error "${name} @${version} requires OS X 10.14 or later."  
        return -code error "incompatible OS X version"  
    }  
}
```

3.22. The `conflicts_build` PortGroup

To specify that a port would fail to build properly if certain other ports are active at configure, build and/or deinstall time, use the `conflicts_build` PortGroup:

```
PortGroup          conflicts_build 1.0  
  
conflicts_build    <conflicting port>
```

Ideally this feature should be integrated into MacPorts base, but it hasn't been done yet.

3.23. The `select` PortGroup

The `select` PortGroup is functionally similar to the Debian alternatives system. It uses symbolic links to let users choose between several alternatives that provide the same CLI command.

To list and select between alternatives, use the `port` command:

```
port select --show python
port select --list python
port select --set python python36
```

The command details are documented in the `port(1)` man page.

To create a set of ports to select from, you should first write a `xxx_select` base port. Check out the `fennel_select` and `lua-fennel` ports for an example.

3.24. Additional CMake module path

For dependency ports like `libfmt9` with non-standard CMake module path, specify the following values to let CMake auto-discover required build flags:

```
depends_lib-append port:libfmt9
cmake.module_path-append \
    ${prefix}/lib/libfmt9/cmake
```

3.25. Override `CMAKE_BUILD_TYPE`

If the upstream project requires specific CMake build type for a proper release build, you can try override the `cmake 1.1` PortGroup default with:

```
cmake.build_type      Release
```

4. Upstream build fixes

This chapter lists some common build script mistakes that needs to be corrected via patching.

Usually you should propose these fixes upstream, and remove the patches once upstream releases a new version containing the fixes.

4.1. Prioritize `LUAJIT_INCLUDE_DIR` over default search path in CMake

In CMake, use the `BEFORE` keyword in `include_directories()` to prioritize an include path:

```
--- CMakeLists.txt.orig 2023-07-02 06:18:57
+++ CMakeLists.txt 2023-12-17 15:07:33
@@ -133,7 +133,7 @@
     if(NOT LUAJIT_INCLUDE_DIR)
         message( FATAL_ERROR "Failed to find LuaJIT headers. Variable
`LUAJIT_INCLUDE_DIR' expected to be defined.")
     endif()
-    include_directories(${LUAJIT_INCLUDE_DIR})
+    include_directories(BEFORE ${LUAJIT_INCLUDE_DIR})
     else (USE_LUAJIT)
         # We only link the libs on Windows, so find_package fully succeeding
         # is only required on Windows
```

4.2. Hard-coded paths in binary

This is usually an exception of the always upstream rule.

When upstream hard-codes paths in C code, you may need to patch it with the prefix the port is built for using the `patchfiles` and `reinplace` combo. For example:

```
patchfiles          patch-ckuus5.c.diff

post-patch {
    reinplace "s|@@prefix@@|${prefix}|g"    ${worksrcpath}/ckuus5.c
}

--- ckuus5.c.orig 2017-04-30 16:29:34.000000000 +0000
+++ ckuus5.c 2019-07-28 03:00:00.000000000 +0000
@@ -950,6 +950,7 @@
    char * k_info_dir = NULL;                                /* Where to find text files */
    #ifdef UNIX
    static char * txtmdir[] = {
+    "@@prefix@@/share/kermi"                                /* Mac OS X MacPorts */
        "/usr/local/doc/",                                    /* Linux, SunOS, ... */
        "/usr/share/lib/",                                    /* HP-UX 10.xx... */
        "/usr/share/doc/",                                    /* Other possibilities... */
```

4.3. Respect `DESTDIR` in Makefile correctly

If upstream forgot to respect `DESTDIR` during `make install`, the file could be installed into `${prefix}` directly without going through port activation.

Fix this by patching the build scripts. For example,

1. Patch direct `mkdir` calls:


```

--- src/Makefile.am.orig 2022-08-23 01:20:20.000000000 +0800
+++ src/Makefile.am 2023-01-15 14:14:01.000000000 +0800
@@ -5,5 +5,5 @@
 AM_CFLAGS = -DSYSCONFDIR=\"$(sysconfdir)\" -DSTATEDIR=\"$(oc_statedir)\"

install-data-hook:
- $(MKDIR_P) $(oc_statedir)
+ $(MKDIR_P) $(DESTDIR)$ (oc_statedir)

```

```

--- launcher/CMakeLists.txt.orig 2025-12-20 01:12:07
+++ launcher/CMakeLists.txt 2025-12-20 01:12:18
@@ -1570,13 +1570,13 @@

    # Add qt.conf - this makes Qt stop looking for things outside the bundle
    install(
-       CODE "file(WRITE \"\${CMAKE_INSTALL_PREFIX}/${RESOURCES_DEST_DIR}/
qt.conf\" \" \")"
+       CODE "file(WRITE \"\${ENV{DESTDIR}}\${CMAKE_INSTALL_PREFIX}/
${RESOURCES_DEST_DIR}/qt.conf\" \" \")"
        COMPONENT bundle
    )
    # Add qtlogging.ini as a config file
    install(
        FILES "qtlogging.ini"
-       DESTINATION "${CMAKE_INSTALL_PREFIX}/${RESOURCES_DEST_DIR}"
+       DESTINATION "${ENV{DESTDIR}}\${CMAKE_INSTALL_PREFIX}/${RESOURCES_DEST_DIR}"
        COMPONENT bundle
    )
endif()

```

```
-- makefile.orig 2017-05-01 12:43:14.000000000 +0000
+++ makefile      2019-07-28 03:00:00.000000000 +0000
@@ -1116,7 +1116,7 @@
     cp $(BINARY) $(DESTDIR)$(BINDIR)/kermit || exit 1;\
     chmod 755    $(DESTDIR)$(BINDIR)/kermit || exit 1;\
     rm -f        $(DESTDIR)$(BINDIR)/kermit-sshsub;\
-    ln -s        $(DESTDIR)$(BINDIR)/kermit\
+    ln -s        kermit\
             $(DESTDIR)$(BINDIR)/kermit-sshsub || exit 1;\
     echo 'set flag=f\nPrC Removing binaries' >&3;\
     echo "RmF $(DESTDIR)$(BINDIR)/kermit-sshsub" >&3;\
```

You may need to rename functions or macros that conflicts with definitions in the system library headers. For example:

```

*prefix, FILE *outf)
    *p = 0;
    fputs(t, outf);
    t = &p[7];
-    if (macosx()) {
+    if (is_macosx()) {
        /* all the same size for roff
        *      #BKMOD#
        */
diff --git src/libc/unix.h src/libc/unix.h
index bfbd1f2..6ffd7cc 100644
--- src/libc/unix.h
+++ src/libc/unix.h
@@ -74,9 +74,9 @@

#ifdef __APPLE__
-#define macosx() 1
+#define is_macosx() 1
    #else
-#define macosx() 0
+#define is_macosx() 0
    #endif

/* tcp/tcp.c */

```

4.5. Add missing `#include`

This usually results in a compilation error of either `-Wimplicit-function-declaration` or undeclared identifiers. For example:

```

Fix:
error: implicit declaration of function 'utime' is invalid in C99 [-Werror,-
Wimplicit-function-declaration]
error: implicit declaration of function 'kill' is invalid in C99 [-Werror,-
Wimplicit-function-declaration]
https://github.com/kholtman/afio/issues/17
--- afio.c.orig 2018-11-30 08:25:04.000000000 -0600
+++ afio.c 2022-01-26 17:24:28.000000000 -0600
@@ -162,6 +162,8 @@

#include <stdio.h>
#include <errno.h>
+#include <utime.h>
+#include <signal.h>

#ifdef sun
#include <sys/types.h>

```

4.6. Remove `-Werror` from build flags

Compilers add more warnings in new versions, so `-Wall -Werror` often causes trouble for downstream packaging work. Strip the `-Werror` in build scripts to workaround it:

```
post-patch {
    reinplace "s|-Werror||g" ${worksrcpath}/CMakeLists.txt
}
```

4.7. Sequential build with `-j1`

MacPorts defaults to build in parallel in the build phase by passing `-j${build.jobs}` option to the `make` command, utilizing multiple CPU cores to speed up the build process.

If the Makefile doesn't properly declare dependencies of each target, you may encounter semi-random build failures. The best solution is to fix the Makefile and propose your changes upstream.

Alternatively, you can use the `use_parallel_build` keyword as a workaround:

```
use_parallel_build no
```

4.8. Preserve specific empty directories in destroot

To preserve a directory even if it's empty upon destroot completion, set `destroot.keepdirs` to its path like so:

```
destroot.keepdirs      ${destroot}${prefix}/etc/haproxy
```

This is useful if users are expected to put their custom configuration files in this directory.

4.9. Fix file permissions in extracted tarballs

For example, to fix directory permissions:

```
post-extract {
    # Fix permissions
    fs-traverse dir ${worksrcpath} {
        if {[file isdirectory ${dir}]} {
            file attributes ${dir} -permissions 0755
        }
    }
}
```

5. Tips

This chapter gives you some tips that could help speed up your development process.

5.1. Livecheck and Repology

As a port maintainer, you can use the following command to livecheck all ports that you maintain:

```
port -v livecheck maintainer:xxx
```

For GitHub project with beta releases, it's often easier to match against the latest release in livecheck:

```
livecheck.url      ${github.homepage}/releases/latest
```

If a port's upstream is inactive or gone, you may skip livecheck for this port:

```
# Upstream is archived.  
livecheck.type      none
```

This is also used in subports to ensure that only the main port could perform livecheck, avoiding duplicate results.

Alternatively, Repology^o monitors port freshness in MacPorts by comparing package versions against other packaging ecosystems and provides both per-maintainer and global views.

5.2. Force building from source

If the current port version (including revision and epoch) is unchanged, `port install` would attempt to fetch and install a binary archive, skipping a source build.

To validate that a port still builds and installs correctly, and avoid unnecessarily building its dependency ports from source, run:

```
sudo port clean <port>  
sudo port destroy <port>  
sudo port -s install <port>
```

While `port clean` is not needed on first run, further changes to the Portfile may require a clean workdir.

5.3. Verify patches quickly

To verify that patches still apply to a new upstream version, run:

```
sudo port clean <port>  
sudo port -v patch <port>
```

and iterate the patches as needed.

If the distfiles are very large, you may manually revert partially applied patches and re-run `port patch` without `port clean` to speed up the process.

5.4. Create a patch file from git commands

Use `git format-patch --no-prefix [ <revision-range> | -<number> ]` to generate patch files that can be used with the default `patch.pre_args ( -p0 )` in MacPorts.

Similarly, you can use `git diff --no-prefix HEAD [ -- <path> ]` to generate a diff for uncommitted changes under `path`.

Alternatively, you could override `patch.pre_args` and use git-style patches directly:

```
patch.pre_args=replace -p0 -p1
```

5.5. Install a subport without re-indexing

To install a subport of the port in current working directory, you may use:

```
sudo port install subport=xxx
```

This helps quickly prototype a port without updating the `PortIndex` files, especially when the `python PortGroup` is used because the main `py-` port doesn't contain anything substantial.

5.6. Switch between conflicting dependencies

If you need to switch between two conflicting library ports, use `port deactivate` and `port activate` to replace the active port. If the two ports aren't ABI-compatible, you may be prompted to rebuild all dependent ports during an automatic `port rev-upgrade`.

5.7. Identify library dependencies

Including library dependencies that are directly referenced by the binary files of a port helps identify revision bumps needed when a SONAME change or ABI-breaking change happens to a library port. To identify such `depends_lib` dependencies for a port, run `otool -L` on all executable files and dynamic library files of the port and use `port provides <path>` to find the relevant dependency ports.

Additionally, use trace mode (`-t`) to let MacPorts report undeclared dependencies based on files accessed:

```
sudo port -vst destrout <port>
```

This won't help with identifying `depends_run` dependencies though, so you should test the program's basic functionality before submitting a PR.